

Python 数值分析 算法实践



Python Numerical Analysis Algorithm Practice

第7章 解线性方程组的迭代法



讲授: X X X



日期: 2024年4月19日

第7章 解线性方程组的迭代法

设线性方程组 $\mathbf{Ax} = \mathbf{b}$, 通过变换获得同解方程组 $\mathbf{x} = \mathbf{Mx} + \mathbf{g}$, 对于初始向量 $\mathbf{x}^{(0)} \in \mathbb{R}^n$, 构造迭代公式

$$\mathbf{x}^{(k+1)} = \mathbf{Mx}^{(k)} + \mathbf{g}, k = 0, 1, \dots,$$

其中 $\mathbf{M}$ 为迭代矩阵, $\mathbf{x}^{(k)}$ 为第 k 次迭代解向量, $\mathbf{g}$ 为常向量. 给定初始迭代解向量 $\mathbf{x}^{(0)}$, 若向量序列 $\{\mathbf{x}^{(k)}\}$ 收敛, 则当 k 充分大时, 将 $\mathbf{x}^{(k)}$ 作为近似解, 这就是迭代法的基本思想.

需要思考的问题:

- (1) 如何构造迭代矩阵 $\mathbf{M}$? 采用不同的方法将得到不同的迭代矩阵.
- (2) 迭代序列是否收敛? 收敛条件是什么? 取决于迭代矩阵的性态, 对于无参数的迭代法, 其收敛的充要条件是迭代矩阵的谱半径小于 1.
- (3) 误差估计及迭代的加速.

解线性方程组的迭代法

```
graph LR; A[解线性方程组的迭代法] --- B[7.1 雅可比迭代法与高斯-赛德尔迭代法]; A --- C[7.2 超松弛迭代法]; A --- D[7.3 共轭梯度法]; A --- E[7.4 * 二维泊松方程边值问题稀疏矩阵迭代求解]; C --- C1[7.2.1 逐次超松弛迭代法]; C --- C2[7.2.2 块迭代法]; D --- D1[7.3.1 最速下降法]; D --- D2[7.3.2 共轭梯度法]; D --- D3[7.3.3 预处理共轭梯度法];
```

7.1 雅可比迭代法与高斯-赛德尔迭代法

7.2 超松弛迭代法

7.2.1 逐次超松弛迭代法

7.2.2 块迭代法

7.3 共轭梯度法

7.3.1 最速下降法

7.3.2 共轭梯度法

7.3.3 预处理共轭梯度法

7.4 * 二维泊松方程边值问题稀疏矩阵迭代求解

7.1 雅可比迭代法与高斯—赛德尔迭代法

雅可比(Jacobi)迭代法的主要思想: 假设系数矩阵 $\mathbf{A}$ 的对角元素 $a_{ii} \neq 0, i = 1, 2, \dots, n$, 对线性方程组进行同解变形, 使第 i 个方程的等号左端只有 x_i , 等号右端不出现 x_i , 进而构造迭代公式:

$$\begin{cases} x_1^{(k+1)} = (-a_{12}x_2^{(k)} - a_{13}x_3^{(k)} - \dots - a_{1n}x_n^{(k)} + b_1)/a_{11}, \\ x_2^{(k+1)} = (-a_{21}x_1^{(k)} - a_{23}x_3^{(k)} - \dots - a_{2n}x_n^{(k)} + b_2)/a_{22}, \\ \vdots \\ x_n^{(k+1)} = (-a_{n1}x_1^{(k)} - a_{n2}x_2^{(k)} - \dots - a_{n,n-1}x_{n-1}^{(k)} + b_n)/a_{nn}. \end{cases}$$

化为更简形式:

$$x_i^{(k+1)} = \left(b_i - \sum_{j=1, j \neq i}^n a_{ij}x_j^{(k)} \right) / a_{ii}, \quad a_{ii} \neq 0, \quad i = 1, 2, \dots, n, \quad k = 0, 1, 2, \dots.$$

7.1 雅可比迭代法与高斯—赛德尔迭代法

将系数矩阵 $\mathbf{A}$ 分解为三部分

$$\mathbf{A} = \mathbf{D} - \mathbf{L} - \mathbf{U} = \begin{pmatrix} a_{11} & & & \\ & a_{22} & & \\ & & \ddots & \\ & & & a_{nn} \end{pmatrix} - \begin{pmatrix} 0 & & & \\ -a_{21} & 0 & & \\ \vdots & \vdots & \ddots & \\ -a_{n1} & -a_{n2} & \cdots & 0 \end{pmatrix} - \begin{pmatrix} 0 & -a_{12} & \cdots & -a_{1n} \\ & 0 & \cdots & -a_{2n} \\ & & \ddots & \vdots \\ & & & 0 \end{pmatrix}$$

可得雅可比迭代法的矩阵表示形式

$$\mathbf{x}^{(k+1)} = \mathbf{B}\mathbf{x}^{(k)} + \mathbf{f}, \quad k = 0, 1, \dots$$

其中 $\mathbf{B} = \mathbf{D}^{-1}(\mathbf{L} + \mathbf{U}) \equiv \mathbf{J}$, $\mathbf{f} = \mathbf{D}^{-1}\mathbf{b}$, $\mathbf{J}$ 称为雅可比迭代法的迭代矩阵. 雅可比迭代法收敛的充要条件是 $\rho(\mathbf{J}) < 1$, 其中 $\rho(\mathbf{J})$ 为 $\mathbf{J}$ 的谱半径, 计算方法为

$$\rho(\mathbf{J}) = \max_{1 \leq i \leq n} |\lambda_i|,$$

其中 λ_i ($i = 1, 2, \dots, n$) 为迭代矩阵 $\mathbf{J}$ 的 n 个特征值.

2. 高斯—赛德尔迭代法

高斯—赛德尔(Gauss-Seidel, G-S)迭代法是雅可比迭代法的改进方法,雅可比迭代收敛不能推出G-S迭代收敛, G-S迭代收敛也不能推出雅可比迭代收敛. G-S迭代法对原方程组的变形与雅可比迭代法相同, 但G-S迭代法的迭代公式与雅可比迭代法不同:

$$\begin{cases} x_1^{(k+1)} = (-a_{12}x_2^{(k)} - a_{13}x_3^{(k)} - a_{14}x_4^{(k)} - \cdots - a_{1,n}x_n^{(k)} + b_1)/a_{11}, \\ x_2^{(k+1)} = (-a_{21}x_1^{(k+1)} - a_{23}x_3^{(k)} - a_{24}x_4^{(k)} - \cdots - a_{2,n}x_n^{(k)} + b_2)/a_{22}, \\ x_3^{(k+1)} = (-a_{31}x_1^{(k+1)} - a_{32}x_2^{(k+1)} - a_{34}x_4^{(k)} - \cdots - a_{3,n}x_n^{(k)} + b_3)/a_{33}, \\ \vdots \\ x_n^{(k+1)} = (-a_{n1}x_1^{(k+1)} - a_{n2}x_2^{(k+1)} - a_{n3}x_3^{(k+1)} - \cdots - a_{n,n-1}x_{n-1}^{(k+1)} + b_n)/a_{nn}. \end{cases}$$

化为更简形式:

$$x_i^{(k+1)} = \left(b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij}x_j^{(k)} \right) / a_{ii}, \quad a_{ii} \neq 0, \quad i=1, 2, \cdots, n, \quad k=0, 1, \cdots.$$

2. 高斯—赛德尔迭代法

假设将系数矩阵 $\mathbf{A}$ 分解为三部分 $\mathbf{A} = \mathbf{D} - \mathbf{L} - \mathbf{U}$, 则高斯—赛德尔迭代法的矩阵表示形式为

$$\mathbf{x}^{(k+1)} = \mathbf{B}\mathbf{x}^{(k)} + \mathbf{f}, k = 0, 1, \dots.$$

其中 $\mathbf{B} = (\mathbf{D} - \mathbf{L})^{-1}\mathbf{U} \equiv \mathbf{G}$, $\mathbf{f} = (\mathbf{D} - \mathbf{L})^{-1}\mathbf{b}$, $\mathbf{G}$ 称为高斯—赛德尔迭代法的迭代矩阵, 收敛的充要条件是 $\rho(\mathbf{G}) < 1$, $\rho(\mathbf{G})$ 为迭代矩阵 $\mathbf{G}$ 的谱半径.

如果方程组为对角占优的, 则其雅可比迭代、高斯—赛德尔迭代对于任意给定初始解向量 $\mathbf{x}^{(0)}$ 均收敛.

7.1 雅可比迭代法与高斯—赛德尔迭代法

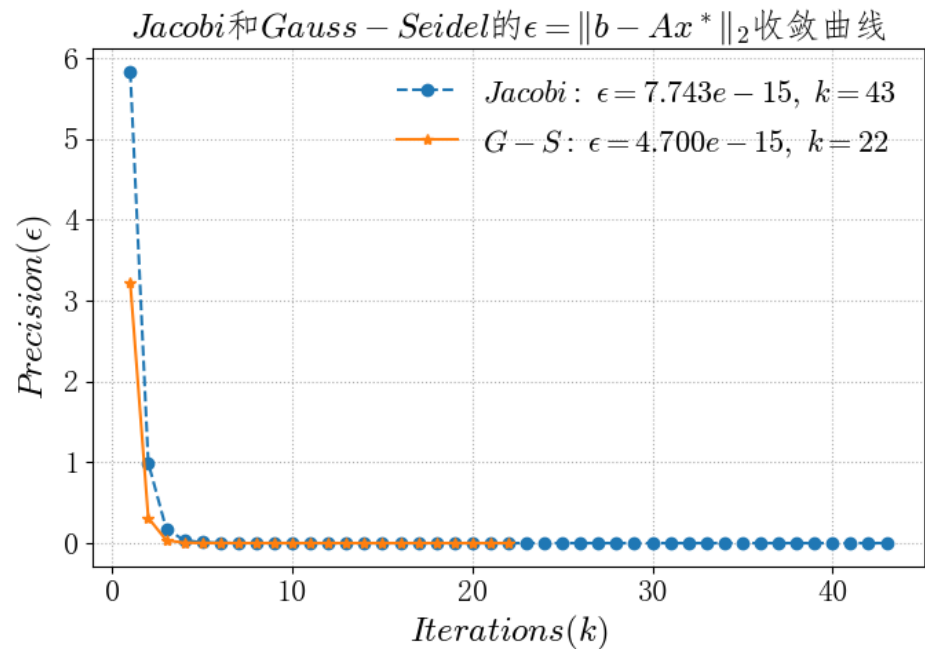
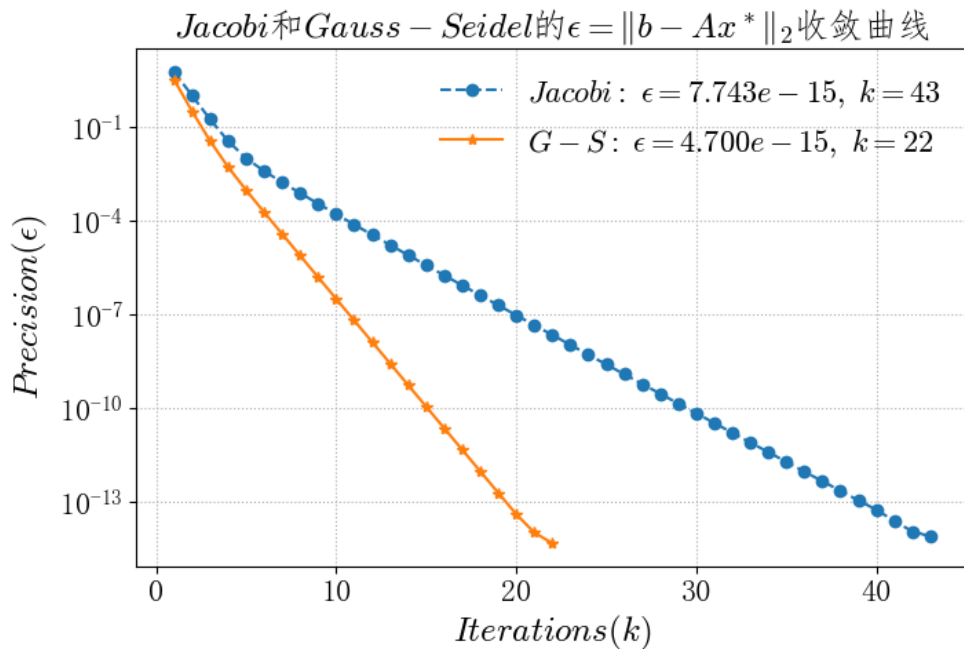
例1: 已知带状方程组, 设初始解向量为 $\mathbf{x}^{(0)} = (0.01, 0.01, \dots, 0.01)^T$, 精度 $\varepsilon = 10^{-14}$, 试采用雅可比迭代法和高斯—赛德尔迭代法求解带状线性方程组满足精度要求的近似解.

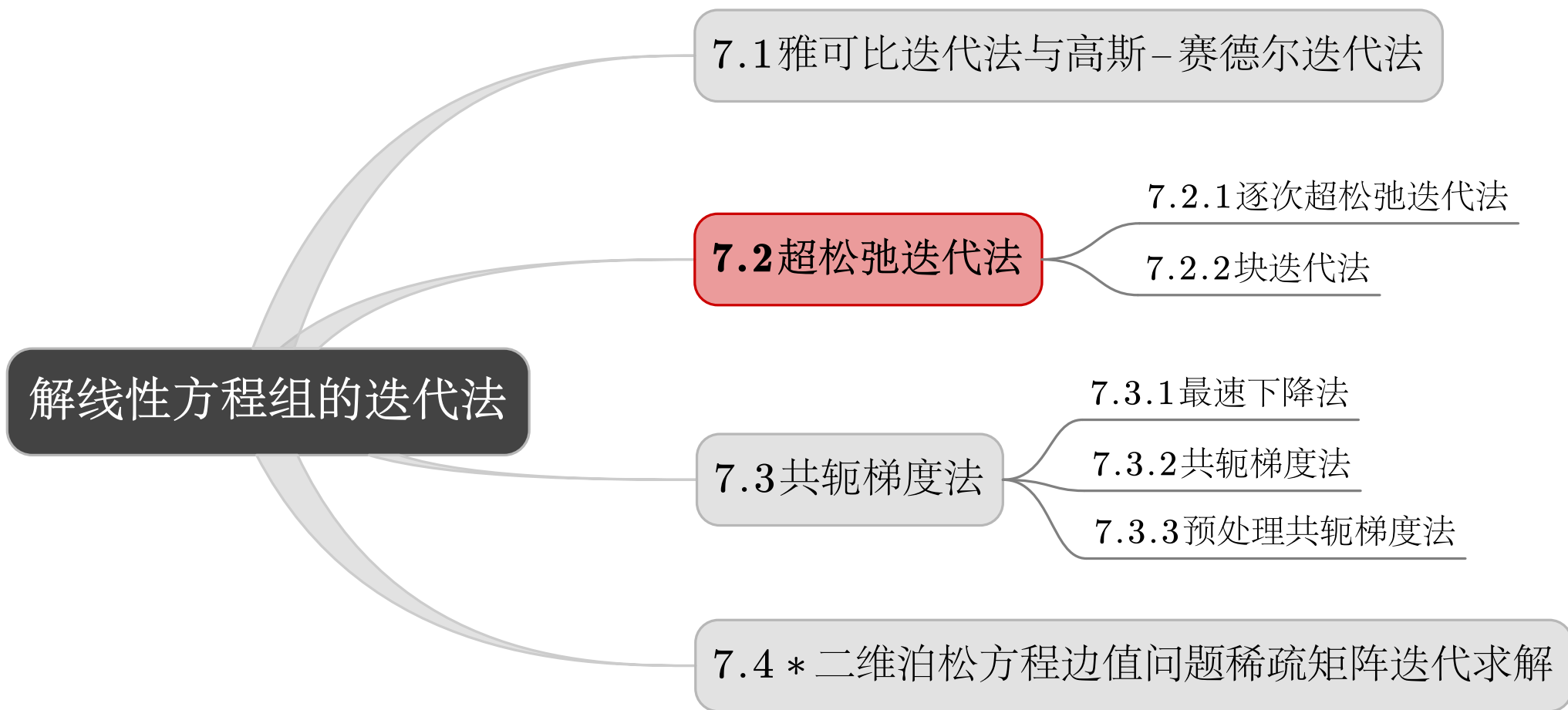
$$\begin{pmatrix} 12 & -2 & 1 & & & & \\ -2 & 12 & -2 & 1 & & & \\ & 1 & -2 & 12 & -2 & 1 & \\ & & 1 & -2 & 12 & -2 & 1 \\ & & & \ddots & \ddots & \ddots & \ddots \\ & & & & 1 & -2 & 12 & -2 & 1 \\ & & & & & 1 & -2 & 12 & -2 \\ & & & & & & 1 & -2 & 12 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ \vdots \\ x_{48} \\ x_{49} \\ x_{50} \end{pmatrix} = \begin{pmatrix} 5 \\ 5 \\ 5 \\ 5 \\ \vdots \\ 5 \\ 5 \\ 5 \end{pmatrix}$$

该带状矩阵方程组的解向量共50个元素, 且解元素的规律性满足 $x_i = x_{50-i}$, $i = 0, 1, 2, \dots, 25$, 即第一个元素与最后一个元素相同, 第2个元素与倒数第二个元素相同, 依次类推,

表 7-1 带状矩阵方程组的解 (前 25 个, 按 Python 输出格式)

k	x_k	k	x_k	k	x_k	k	x_k	k	x_k
1	0.46379552	6	0.49998535	11	0.50000011	16	0.5	21	0.5
2	0.53728461	7	0.50008872	12	0.5000002	17	0.5	22	0.5
3	0.50902292	8	0.50001532	13	0.50000002	18	0.5	23	0.5
4	0.49822163	9	0.49999479	14	0.49999999	19	0.5	24	0.5
5	0.49894186	10	0.49999786	15	0.50	20	0.5	25	0.5





7.2 超松弛迭代法

超松弛迭代是求解大型稀疏方程组的一种有效方法. 松弛过程公式:

$$\begin{cases} \tilde{x}_i^{(k+1)} = \left(b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij} x_j^{(k)} \right) / a_{ii}, \\ x_i^{(k+1)} = \omega \tilde{x}_i^{(k+1)} + (1 - \omega) x_i^{(k)}. \end{cases}$$

当 $\omega < 1$ 时为低松弛, 当 $\omega = 1$ 时为高斯—赛德尔迭代法. 由于新值 $\tilde{x}_i^{(k+1)}$ 通常优于旧值 $x_i^{(k)}$, 在将两者加工成松弛值 $x_i^{(k+1)}$ 时, 自然要求松弛因子 $\omega > 1$, 以尽量发挥新值的优势. 这类松弛迭代称作超松弛法, 简称SOR方法(Successive Over-Relaxation, SOR).

将迭代与松弛归并得:

$$x_i^{(k+1)} = x_i^{(k)} + \frac{\omega}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i}^n a_{ij} x_j^{(k)} \right), \quad i = 1, 2, \dots, n, \quad k = 0, 1, \dots$$

7.2 超松弛迭代法

使用SOR方法的关键在于选取合适的松弛因子. 松弛因子的取值对收敛速度影响很大. 实际计算时, 通常依据系数矩阵的特点, 并结合科学计算的实践经验来选取合适的松弛因子. 特别地, 当 $\mathbf{A}$ 为对称正定的三对角矩阵时, 有

$$\omega_{\text{opt}} = \frac{2}{1 + \sqrt{1 - (\rho(\mathbf{J}))^2}},$$

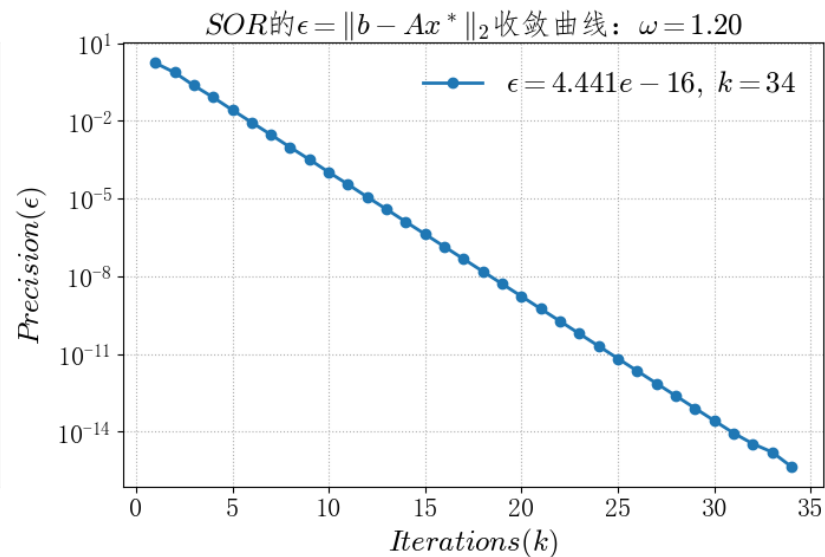
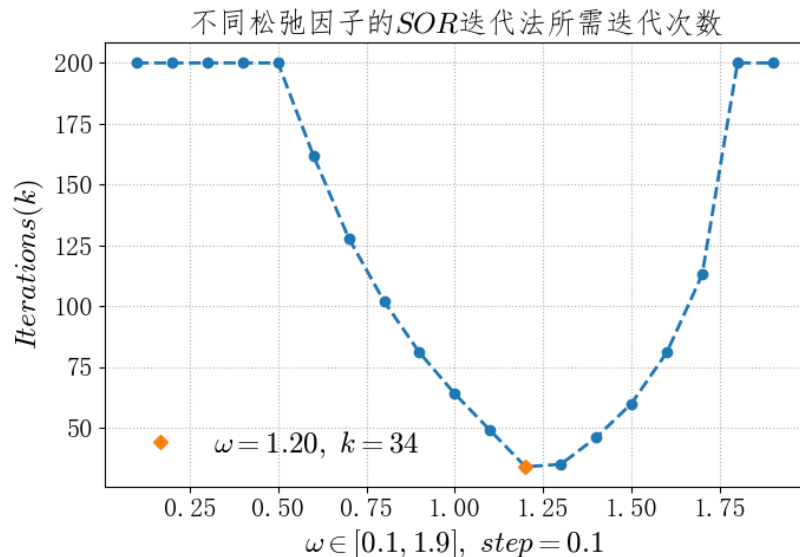
其中 $\rho(\mathbf{J})$ 为雅可比迭代矩阵 $\mathbf{J}$ 的谱半径.

设 $\mathbf{Ax} = \mathbf{b}$, 如果 $\mathbf{A}$ 为对称正定矩阵, 当 $0 < \omega < 2$ 时, 则解 $\mathbf{Ax} = \mathbf{b}$ 的SOR迭代法收敛. 如果 $\mathbf{A}$ 为严格对角占优矩阵, 当 $0 < \omega \leq 1$ 时, 则 $\mathbf{Ax} = \mathbf{b}$ 解的SOR迭代法收敛. 当 $1 < \omega < 2$ 时, 解的SOR迭代法可能收敛.

7.2 超松弛迭代法

例2:用SOR迭代法求解如下线性方程组, 并选择最优的松弛因子 $\omega \in [0.1, 1.9]$, 精度要求 $\varepsilon = 10^{-15}$, 最大迭代次数 200, 初始解向量为 $\mathbf{x}^{(0)} = (0, 0, 0, 0)^T$.

$$\begin{pmatrix} -4 & 1 & 1 & 1 \\ 1 & -4 & 1 & 1 \\ 1 & 1 & -4 & 1 \\ 1 & 1 & 1 & -4 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \end{pmatrix}$$



7.2 超松弛迭代法

表 7-2 不同的超松弛因子 ω 下的收敛精度 (最大迭代次数 200, $k = 1, 2, \dots, 19$)

ω_k	$\ b - Ax^*\ _2$	ω_k	$\ b - Ax^*\ _2$	ω_k	$\ b - Ax^*\ _2$
0.1	0.01035270588602879	0.8	7.021666937153402e-16	1.5	4.440892098500626e-16
0.2	2.961144275909281e-05	0.9	6.280369834735101e-16	1.6	9.930136612989092e-16
0.3	4.148345637495417e-08	1.0	6.280369834735101e-16	1.7	4.440892098500626e-16
0.4	2.426515995179419e-11	1.1	6.280369834735101e-16	1.8	1.884110950420530e-15
0.5	4.636427468134552e-15	1.2	4.440892098500626e-16	1.9	1.477517828767679e-08
0.6	7.691850745534255e-16	1.3	4.440892098500626e-16		
0.7	7.691850745534255e-16	1.4	4.440892098500626e-16		

解线性方程组的迭代法

```
graph LR; A[解线性方程组的迭代法] --- B[7.1 雅可比迭代法与高斯-赛德尔迭代法]; A --- C[7.2 超松弛迭代法]; A --- D[7.3 共轭梯度法]; A --- E[7.4 * 二维泊松方程边值问题稀疏矩阵迭代求解]; D --- F[7.3.1 最速下降法]; D --- G[7.3.2 共轭梯度法]; D --- H[7.3.3 预处理共轭梯度法];
```

7.1 雅可比迭代法与高斯-赛德尔迭代法

7.2 超松弛迭代法

7.2.1 逐次超松弛迭代法

7.2.2 块迭代法

7.3 共轭梯度法

7.3.1 最速下降法

7.3.2 共轭梯度法

7.3.3 预处理共轭梯度法

7.4 * 二维泊松方程边值问题稀疏矩阵迭代求解

7.3 共轭梯度法

梯度法相对于通过对系数矩阵简单分裂构造的迭代法不同, 无需考虑收敛的问题, 而且求解的速度也相对较快. 共轭梯度法简称CG (conjugate gradient)方法, 又称共轭斜量法, 它是一种变分方法, 对应于求一个二次函数的极值.

设 $\mathbf{A} = (a_{ij}) \in \mathbb{R}^{n \times n}$ 是对称正定矩阵, $\mathbf{b} = (b_1, b_2, \dots, b_n)^T$, 求解线性方程组 $\mathbf{Ax} = \mathbf{b}$. 考虑如下定义的二次函数 $\varphi: \mathbb{R}^n \rightarrow \mathbb{R}$:

$$\varphi(\mathbf{x}) = \frac{1}{2}(\mathbf{Ax}, \mathbf{x}) - (\mathbf{b}, \mathbf{x}) = \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n a_{ij} x_i x_j - \sum_{j=1}^n b_j x_j.$$

7.3 共轭梯度法

函数 $\varphi(\mathbf{x})$ 有如下性质:

(1) 对一切 $\mathbf{x} \in \mathbb{R}^n$, $\varphi(\mathbf{x})$ 的梯度 $\nabla \varphi(\mathbf{x}) = \mathbf{Ax} - \mathbf{b}$, 故求 $\mathbf{Ax} - \mathbf{b} = \mathbf{0}$ 的解等价于 $\varphi(\mathbf{x})$ 最小值问题.

(2) 对一切 $\mathbf{x}, \mathbf{y}$ 及, 有

$$\varphi(\mathbf{x} + \alpha \mathbf{y}) = \frac{1}{2}(\mathbf{A}(\mathbf{x} + \alpha \mathbf{y}), \mathbf{x} + \alpha \mathbf{y}) - (\mathbf{b}, \mathbf{x} + \alpha \mathbf{y}) = \varphi(\mathbf{x}) + \alpha(\mathbf{Ax} - \mathbf{b}, \mathbf{y}) + \frac{\alpha^2}{2}(\mathbf{Ay}, \mathbf{y}). \quad (7-12)$$

(3) 设 $\mathbf{x}^* = \mathbf{A}^{-1}\mathbf{b}$ 是线性方程组的解, 则有 $\varphi(\mathbf{x}^*) = -\frac{1}{2}(\mathbf{b}, \mathbf{A}^{-1}\mathbf{b}) = -\frac{1}{2}(\mathbf{Ax}^*, \mathbf{x}^*)$, 且对一切

$\mathbf{x} \in \mathbb{R}^n$, 有

$$\varphi(\mathbf{x}) - \varphi(\mathbf{x}^*) = \frac{1}{2}(\mathbf{Ax}, \mathbf{x}) - (\mathbf{Ax}^*, \mathbf{x}) + \frac{1}{2}(\mathbf{Ax}^*, \mathbf{x}^*) = \frac{1}{2}(\mathbf{A}(\mathbf{x} - \mathbf{x}^*), (\mathbf{x} - \mathbf{x}^*)). \quad (7-13)$$

对于对称正定方阵 $\mathbf{A}$, 求线性方程组 $\mathbf{Ax} = \mathbf{b}$ 等价于求 $\mathbf{x}^* \in \mathbb{R}^n$ 使 $\varphi(\mathbf{x})$ 达到最小值的变分问题.

求解方法是构造一个向量序列 $\{\mathbf{x}^{(k)}\}$, 使 $\varphi(\mathbf{x}^{(k)}) \rightarrow \varphi(\mathbf{x}^*)$.

7.3.1 最速下降法算法

最速下降法是求解无约束最优化问题的算法，可用于求解的解 $\mathbf{Ax} = \mathbf{b}$. 令 $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha_k \mathbf{p}^{(k)}$, 使

$$\varphi(\mathbf{x}^{(k+1)}) = \min_{\alpha \in \mathbb{R}} \varphi(\mathbf{x}^{(k)} + \alpha \mathbf{p}^{(k)}), \quad k = 0, 1, \dots. \quad (7-14)$$

从初始向量 $\mathbf{x}^{(0)}$ 出发，确定一个搜索的方向及搜索步长，循环找到极小值. 具体方法如下：

(1) 最速下降法采取的搜索方向是当前点的负梯度方向，负梯度方向是当前点下降最快的方向，记残差向量

$$\mathbf{r}^{(k)} = -(\mathbf{Ax}^{(k)} - \mathbf{b}),$$

即是函数 $\varphi(\mathbf{x}^{(k)})$ 的负梯度方向

$$\mathbf{p}^{(k)} = -\nabla \varphi(\mathbf{x}^{(k)}) = -(\mathbf{Ax}^{(k)} - \mathbf{b}) = \mathbf{r}^{(k)}.$$

7.3.1 最速下降法算法

(2)每次的搜索步长都使得取极小值.由式(7-14)和式(7-12),欲求目标函数的极值问题,则对求一阶导数并等于0,即

$$\frac{d\varphi(\mathbf{x}^{(k)} + \alpha \mathbf{p}^{(k)})}{d\alpha} = (\mathbf{Ax}^{(k)} - \mathbf{b}, \mathbf{p}^{(k)}) + \alpha (\mathbf{Ap}^{(k)}, \mathbf{p}^{(k)}) = 0 \Rightarrow \alpha_k = - \frac{(\mathbf{Ax}^{(k)} - \mathbf{b}, \mathbf{p}^{(k)})}{(\mathbf{Ap}^{(k)}, \mathbf{p}^{(k)})} = - \frac{(\mathbf{r}^{(k)}, \mathbf{r}^{(k)})}{(\mathbf{Ar}^{(k)}, \mathbf{r}^{(k)})}.$$

综上,具体求解方程组 $\mathbf{Ax} = \mathbf{b}$ 的最速下降法迭代公式(对 $k = 0, 1, \dots$):

$$\begin{cases} \mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha_k \mathbf{r}^{(k)}, \\ \mathbf{r}^{(k)} = -(\mathbf{Ax}^{(k)} - \mathbf{b}), \\ \alpha_k = \frac{(\mathbf{r}^{(k)}, \mathbf{r}^{(k)})}{(\mathbf{Ar}^{(k)}, \mathbf{r}^{(k)})}. \end{cases}$$

当系数矩阵为病态矩阵时,最速下降法收敛速度极慢.

7.3.1 最速下降法算法

例4:利用最速下降法求解如下方程组的解,精度要求 $\varepsilon = 10^{-15}$,最大迭代次数200,初始解向量 $\mathbf{x}^{(0)} = (0, 0, \dots, 0)^T$.

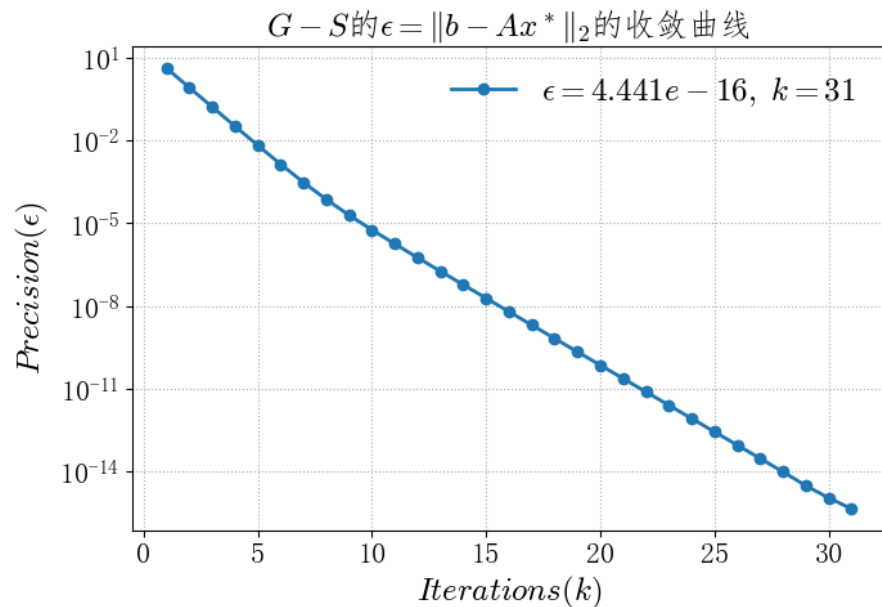
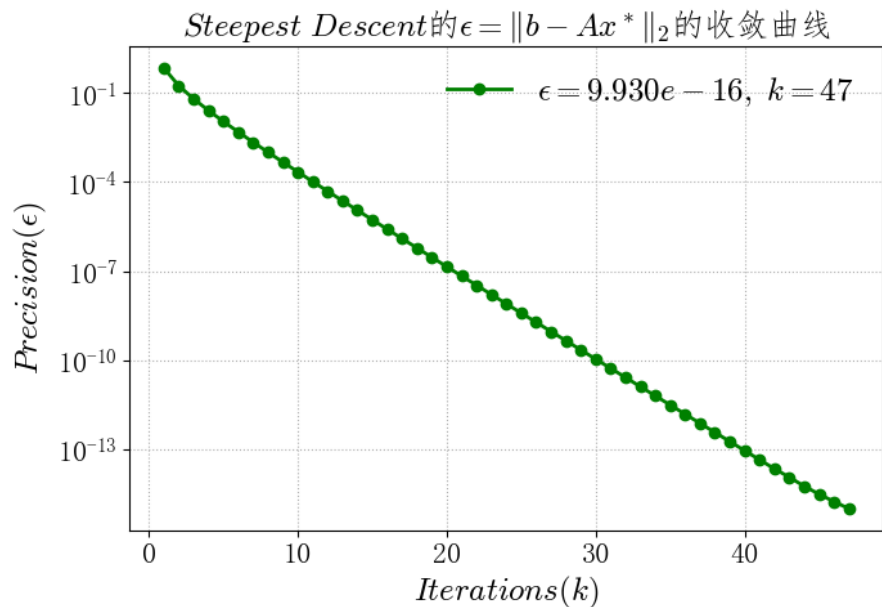
$$\begin{pmatrix} 4 & 1 & & & \\ & 1 & 4 & & \\ & & \ddots & \ddots & \ddots \\ & & & 1 & 4 & 1 \\ & & & & 1 & 4 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_{49} \\ x_{50} \end{pmatrix} = \begin{pmatrix} 3 \\ 3 \\ \vdots \\ 3 \\ 3 \end{pmatrix}$$

矩阵条件数为2.9924276771553098,需迭代47次,如左图所示,验证精度如图例.解向量的元素满足规律: $x_i = x_{50-i}, i = 0, 1, 2, \dots, 25$,如表所示(前25个解元素).而采用高斯—塞德尔迭代法,谱半径 $\rho = 0.24905$,迭代31次满足精度要求,如右图所示,验证精度如图例.

7.3.1 最速下降法算法

表 7-3 最速下降法求解三对角矩阵方程组的解 (前 25 个)

k	x_k	k	x_k	k	x_k	k	x_k	k	x_k
1	0.6339746	6	0.49981495	11	0.50000026	16	0.5	21	0.5
2	0.46410162	7	0.50004958	12	0.49999993	17	0.5	22	0.5
3	0.50961894	8	0.49998671	13	0.50000002	18	0.5	23	0.5
4	0.49742261	9	0.50000356	14	0.5	19	0.5	24	0.5
5	0.50069061	10	0.49999905	15	0.5	20	0.5	25	0.5



7.3.2 共轭梯度算法

共轭梯度法(Conjugate Gradient, CG)是一种求解大型稀疏对称正定方程组十分有效的方法。共轭梯度法是从整体来寻找最佳的搜索方向。

CG的思路: 第一步仍然取负梯度方向作为搜索方向 $\mathbf{p}^{(0)} = \mathbf{r}^{(0)}$, 搜索步长 α_0 使 $\varphi(\mathbf{x}^{(1)})$ 得取极小值; 对后续各步 $k = 1, 2, \dots$, 在过当前点由负梯度向量 $\mathbf{r}^{(k)}$ 和上一步的搜索向量 $\mathbf{p}^{(k-1)}$ 组成的平面内寻找最佳的搜索方向, 即 $\mathbf{r}^{(k)}$ 和 $\mathbf{p}^{(k-1)}$ 的线性组合 $\mathbf{p}^{(k)} = \mathbf{r}^{(k)} + \beta_{k-1} \mathbf{p}^{(k-1)}$, β_{k-1} 为一标量值, 可得更新公式为 $\mathbf{x}^{(k)} = \mathbf{x}^{(k-1)} + \alpha_{k-1} \mathbf{p}^{(k-1)}$. 因此共轭梯度法比最速下降法收敛速度更快。

7.3.2 共轭梯度算法

对于一组向量序列 $\{\mathbf{p}^{(k)}\}_{k=0}^{k=m} \in \mathbb{R}^n$, 若任意两个向量满足

$$(\mathbf{A}\mathbf{p}^{(i)}, \mathbf{p}^{(j)}) = 0, i \neq j, i, j = 0, 1, \dots, m,$$

则称这组向量为 $\mathbb{R}^n$ 中一个 **A-共轭向量组** 或称 **A-正交向量组**. 共轭梯度法迭代生成的搜索方向构成的向量组 $\{\mathbf{p}^{(k)}\}_{k=0}^{k=m} \in \mathbb{R}^n$ 是 **A-共轭向量组**. 如果下次更新的最佳搜索方向为

$$\mathbf{p}^{(k+1)} = \mathbf{r}^{(k+1)} + \beta_k \mathbf{p}^{(k)},$$

则通过 $(\mathbf{A}\mathbf{p}^{(k+1)}, \mathbf{p}^{(k)}) = (\mathbf{A}\mathbf{r}^{(k+1)}, \mathbf{p}^{(k)}) + \beta_k (\mathbf{p}^{(k)}, \mathbf{A}\mathbf{p}^{(k)}) = 0$ 可得

$$\beta_k = -(\mathbf{r}^{(k+1)}, \mathbf{A}\mathbf{p}^{(k)}) / (\mathbf{p}^{(k)}, \mathbf{A}\mathbf{p}^{(k)}).$$

而残差向量 $\mathbf{r}^{(k+1)} = \mathbf{b} - \mathbf{A}\mathbf{x}^{(k+1)} = \mathbf{b} - \mathbf{A}(\mathbf{x}^{(k)} + \alpha_k \mathbf{p}^{(k)}) = \mathbf{r}^{(k)} - \alpha_k \mathbf{A}\mathbf{p}^{(k)}, k = 0, 1, \dots$.

7.3.2 共轭梯度算法

共轭梯度法CG算法步骤:

(1) 任取初始解向量 $\mathbf{x}^{(0)} \in \mathbb{R}^n$, 计算残差向量 $\mathbf{r}^{(0)} = \mathbf{b} - \mathbf{A}\mathbf{x}^{(0)}$, 令搜索方向为 $\mathbf{p}^{(0)} = \mathbf{r}^{(0)}$.

(2) 对 $k = 0, 1, \dots$, 计算

$$\left\{ \begin{array}{l} \text{计算搜索步长: } \alpha_k = \frac{(\mathbf{r}^{(k)}, \mathbf{r}^{(k)})}{(\mathbf{p}^{(k)}, \mathbf{A}\mathbf{p}^{(k)})}, \\ \text{更新解向量: } \mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha_k \mathbf{p}^{(k)}, \\ \text{计算新的梯度: } \mathbf{r}^{(k+1)} = \mathbf{r}^{(k)} - \alpha_k \mathbf{A}\mathbf{p}^{(k)}, \\ \text{计算组合系数: } \beta_k = \frac{(\mathbf{r}^{(k+1)}, \mathbf{r}^{(k+1)})}{(\mathbf{r}^{(k)}, \mathbf{r}^{(k)})}, \\ \text{计算共轭方向: } \mathbf{p}^{(k+1)} = \mathbf{r}^{(k+1)} + \beta_k \mathbf{p}^{(k)}. \end{array} \right.$$

(3) 若 $\mathbf{r}^{(k)} = \mathbf{0}$ 或 $(\mathbf{p}^{(k)}, \mathbf{A}\mathbf{p}^{(k)}) = 0$, 计算停止, 满足精度的解向量 $\mathbf{x}^* = \mathbf{x}^{(k)}$. 由于 $\mathbf{A}$ 正定, 故当

$(\mathbf{p}^{(k)}, \mathbf{A}\mathbf{p}^{(k)}) = 0$ 时, $\mathbf{p}^{(k)} = \mathbf{0}$, 而 $(\mathbf{r}^{(k)}, \mathbf{r}^{(k)}) = (\mathbf{r}^{(k)}, \mathbf{p}^{(k)}) = 0$, 也即 $\mathbf{r}^{(k)} = \mathbf{0}$.

7.3.2 共轭梯度算法

由于 $\{\mathbf{r}^{(k)}\}$ 互相正交, 故在 $\mathbf{r}^{(0)}, \mathbf{r}^{(1)}, \dots, \mathbf{r}^{(n)}$ 中至少有一个零向量. 若 $\mathbf{r}^{(k)} = \mathbf{0}$, 则 $\mathbf{x}^{(k)} = \mathbf{x}^*$. 所以用CG算法求解 n 维线性方程组, 理论上最多 n 步便可求得精确解, 从这个意义上讲CG算法是一种直接法. 但在舍入误差存在的情况下, 很难保证 $\{\mathbf{r}^{(k)}\}$ 的正交性, 此外当 n 很大时, 实际计算步数 $k \ll n$, 即可达到精度要求而不必计算 n 步. 从这个意义上讲, 它是一个迭代法, 所以也有收敛性问题.

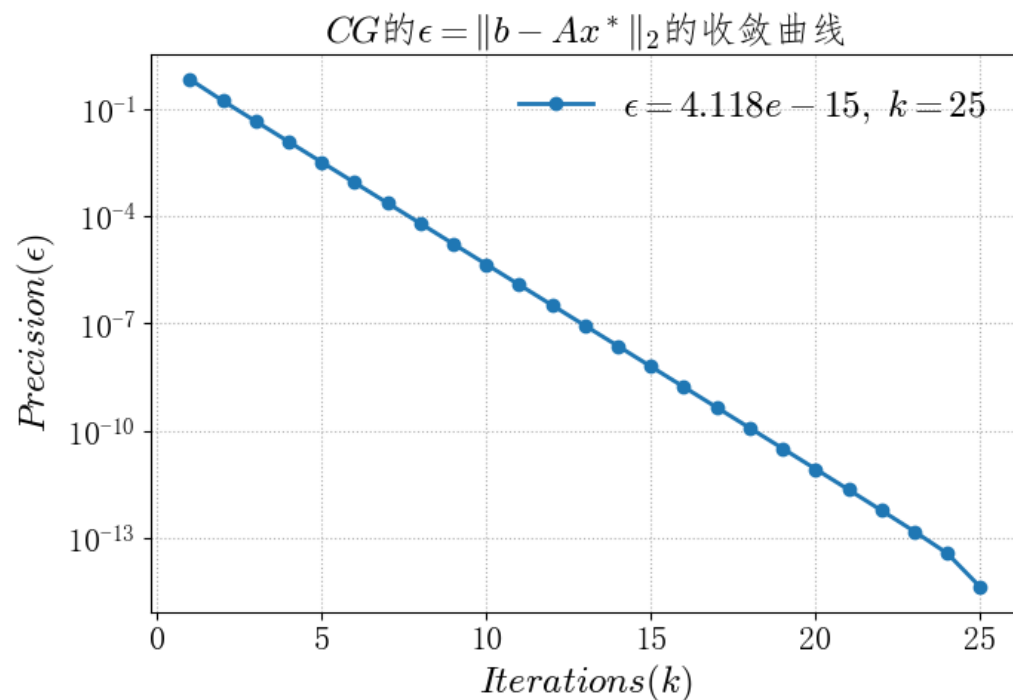
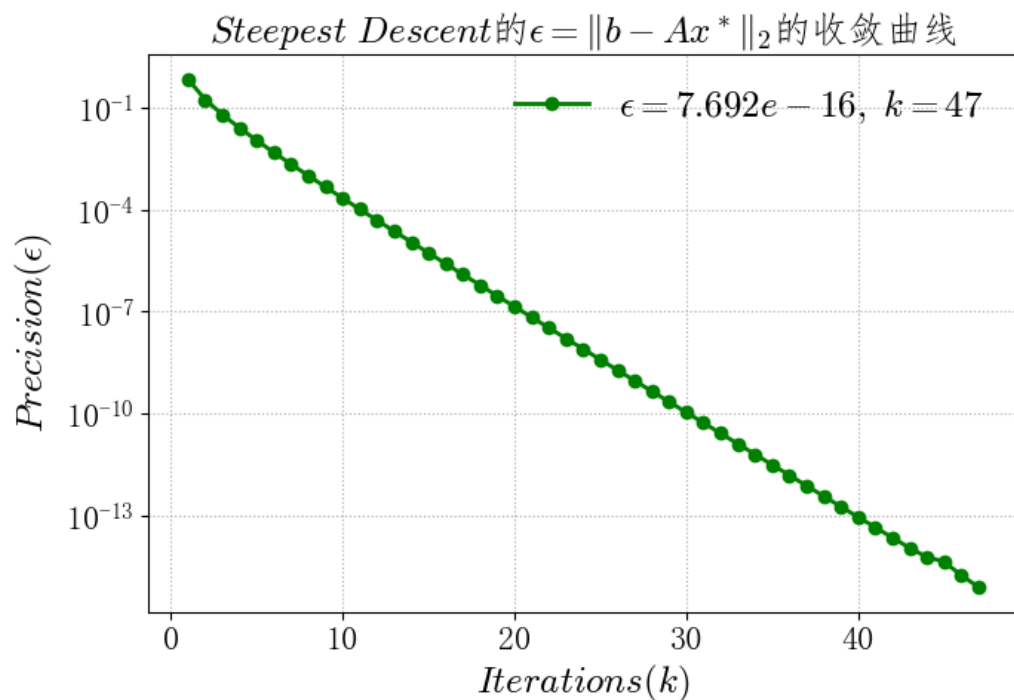
CG算法估计式

$$\|\mathbf{x}^{(k)} - \mathbf{x}^*\|_{\mathbf{A}} \leq 2 \left(\frac{\sqrt{K} - 1}{\sqrt{K} + 1} \right)^k \|\mathbf{x}^{(0)} - \mathbf{x}^*\|_{\mathbf{A}}, \quad (7-17)$$

其中 $\|\mathbf{x}\|_{\mathbf{A}} = \sqrt{(\mathbf{x}, \mathbf{A}\mathbf{x})}$, $K = \text{cond}(\mathbf{A})_2$.

7.3.2 共轭梯度算法

针对例4示例, 采用共轭梯度法求解, 矩阵的条件数 2.9924276771553098 . 图为共轭梯度法和最速下降法解的收敛速度曲线. 可见系数矩阵为良态的对称正定时, 共轭梯度法收敛速度较快.



7.3.3 预处理共轭梯度算法

由CG算法估计式(7-17), $\text{cond}(\mathbf{A})_2$ 为 $\mathbf{A}$ 谱条件数, 也即 $\text{cond}(\mathbf{A})_2 = \lambda_1(\mathbf{A})/\lambda_n(\mathbf{A})$, 当 $\lambda_1 \gg \lambda_n$ 时, 或者当 $\text{cond}(\mathbf{A})_2$ 较大时, 共轭梯度法的收敛效率会变得比较低. 简而言之, 当 $\mathbf{Ax} = \mathbf{b}$ 为病态方程组时, 共轭梯度法会收敛得较慢. 预处理技术是在用共轭梯度法求解之前对系数矩阵做一些变换, 即把 $\mathbf{Ax} = \mathbf{b}$ 转化为等价的 $\tilde{\mathbf{A}}\tilde{\mathbf{x}} = \tilde{\mathbf{b}}$, 使得在同解情况下, 降低系数矩阵 $\tilde{\mathbf{A}}$ 的条件数, 然后再迭代求解.

令 $\tilde{\mathbf{x}} = \mathbf{Cx}$, 针对 $\tilde{\mathbf{x}}$ 的优化目标函数为

$$\varphi(\tilde{\mathbf{x}}) = \frac{1}{2} \tilde{\mathbf{x}}^T (\mathbf{C}^{-T} \mathbf{A} \mathbf{C}^{-1}) \tilde{\mathbf{x}} - (\mathbf{C}^{-1} \mathbf{b})^T \tilde{\mathbf{x}} = \frac{1}{2} (\tilde{\mathbf{A}} \tilde{\mathbf{x}}, \tilde{\mathbf{x}}) - (\tilde{\mathbf{b}}, \tilde{\mathbf{x}}),$$

则 $\tilde{\mathbf{A}} = \mathbf{C}^{-T} \mathbf{A} \mathbf{C}^{-1}$, $\tilde{\mathbf{b}} = \mathbf{C}^{-1} \mathbf{b}$. 如果能找到合适的 $\mathbf{C}$, 使得 $\tilde{\mathbf{A}}$ 具有比较集中的特征值以及较小的条件数, 那么就能提高 $\varphi(\tilde{\mathbf{x}})$ 的收敛效率. 实际中, 得到预处理矩阵 $\mathbf{C}$ 后, 构造一个 $\mathbf{M} = \mathbf{C}^T \mathbf{C}$ 来隐式地完成预处理过程, $\mathbf{M}$ 即为预处理矩阵.

7.3.3 预处理共轭梯度算法

预处理共轭梯度法(preconditioned conjugated gradient method, PCG)算法步骤:

(1) 任取 $\mathbf{x}^{(0)} \in \mathbb{R}^n$, 计算 $\mathbf{r}^{(0)} = \mathbf{b} - \mathbf{A}\mathbf{x}^{(0)}$, 预处理残差向量 $\mathbf{z}^{(0)} = \mathbf{M}^{-1}\mathbf{r}^{(0)}$, 搜索方向为 $\mathbf{p}^{(0)} = \mathbf{z}^{(0)}$.

(2) 对 $k = 0, 1, \dots$, 计算

$$\left\{ \begin{array}{l} \alpha_k = \frac{(\mathbf{r}^{(k)}, \mathbf{z}^{(k)})}{(\mathbf{p}^{(k)}, \mathbf{A}\mathbf{p}^{(k)})}, \\ \mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha_k \mathbf{p}^{(k)}, \\ \mathbf{r}^{(k+1)} = \mathbf{r}^{(k)} - \alpha_k \mathbf{A}\mathbf{p}^{(k)}, \\ \mathbf{z}_{k+1} = \mathbf{M}^{-1}\mathbf{r}_{k+1}, \\ \beta_k = \frac{(\mathbf{r}^{(k+1)}, \mathbf{z}^{(k+1)})}{(\mathbf{r}^{(k)}, \mathbf{z}^{(k)})}, \\ \mathbf{p}^{(k+1)} = \mathbf{z}^{(k+1)} + \beta_k \mathbf{p}^{(k)}. \end{array} \right.$$

(3) 若 $\mathbf{r}^{(k)} = \mathbf{0}$ 或 $(\mathbf{p}^{(k)}, \mathbf{A}\mathbf{p}^{(k)}) = 0$, 计算停止, 则 $\mathbf{x}^{(k)} = \mathbf{x}^*$.

7.3.3 预处理共轭梯度算法

预处理对收敛性的改善主要是通过矩阵 $\mathbf{M}$ 实现的, 适当的 $\mathbf{M}$, 能很好地改善系数矩阵的条件数, 因此 $\mathbf{M}$ 的选择尤为重要. 一般情况, $\mathbf{M}$ 有以下三种选择方式:

(1) 选取 $\mathbf{M}$ 为 $\mathbf{A}$ 的对角元素组成的对角阵, 这种方法简单, 但收敛效果不理想;

(2) 将 $\mathbf{A}$ 作不完全Cholesky分解为 $\mathbf{A} = \mathbf{LL}^T - \mathbf{R}$, $\mathbf{R}$ 为剩余矩阵, 取 $\mathbf{M} = \mathbf{LL}^T$, 此种方法是比较常用的ICCG法, 即不完全Cholesky分解预处理共轭梯度法;

(3) 取 $\mathbf{M}$ 为对称超松弛迭代法SSOR的预处理阵

$$\mathbf{M} = (2 - \omega)^{-1}(\mathbf{D}/\omega + \mathbf{L})(\mathbf{D}/\omega)^{-1}(\mathbf{D}/\omega + \mathbf{L})^T,$$

经过预处理, 能够显著改善收敛速度, 也取决于 ω 的选择.

7.3.3 预处理共轭梯度算法

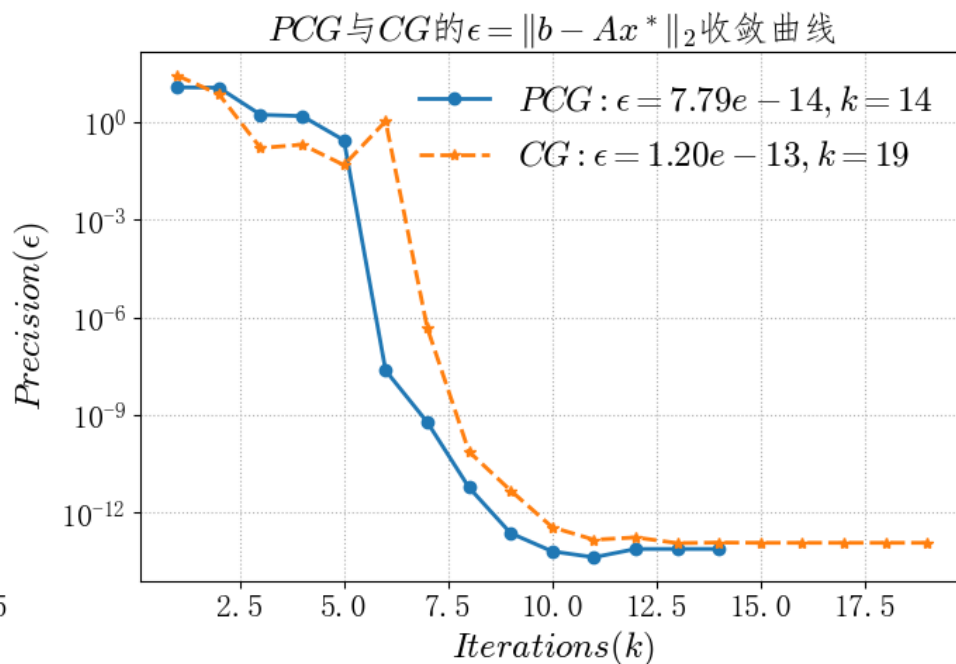
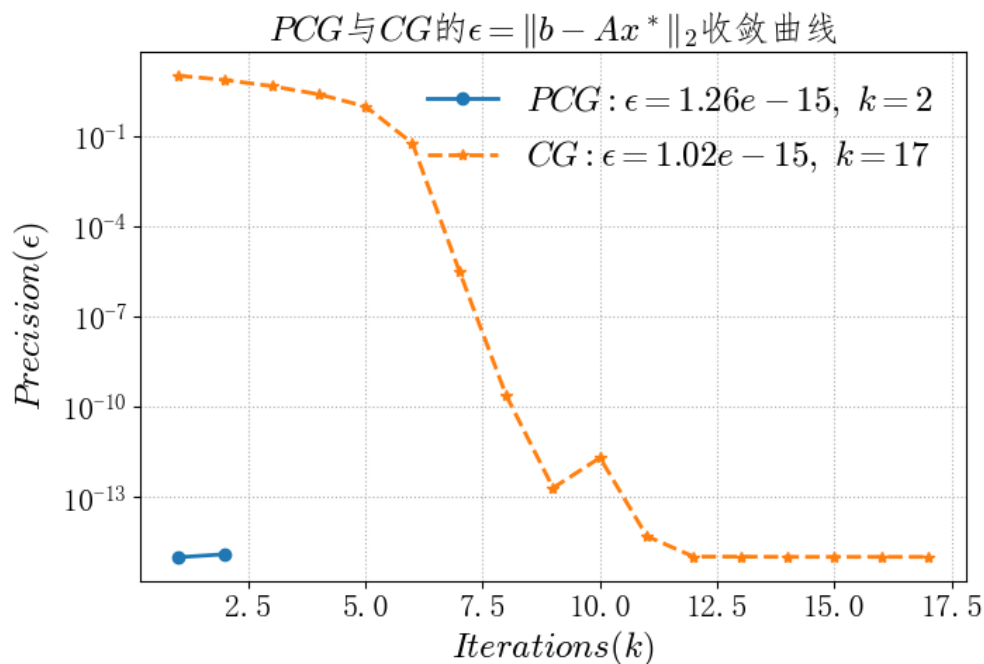
例6:用预处理共轭梯度法求解如下方程组,并与共轭梯度法比较. 精度要求 $\varepsilon = 10^{-16}$, $\omega = 1.5$, 初始解向量 $\mathbf{x}^{(0)} = (0, 0, \dots, 0)^T$, 默认采用SSOR预处理方法.

$$(1) \begin{pmatrix} 1 & & & & & \\ & 10 & & & & \\ & & 10^2 & & & \\ & & & 10^3 & & \\ & & & & 10^4 & \\ & & & & & 10^5 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \end{pmatrix}, (2) \begin{pmatrix} 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 & 5 & 6 \\ 1 & 3 & 6 & 10 & 15 & 21 \\ 1 & 4 & 10 & 20 & 35 & 56 \\ 1 & 5 & 15 & 35 & 70 & 126 \\ 1 & 6 & 21 & 56 & 126 & 252 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \end{pmatrix} = \begin{pmatrix} 6 \\ 21/2 \\ 14 \\ 63/4 \\ 63/4 \\ 113/8 \end{pmatrix}.$$

两个系数矩阵均为病态矩阵, 条件数分别为100000.0和110786.7. 对于方程组(1), 预处理共轭梯度法收敛速度极快, 几乎相当于直接求解, 如左图所示, 两种方法的解向量均为

$$\mathbf{x}^* = (1.000, 0.200, 0.0300, 4.000 \times 10^{-3}, 5.000 \times 10^{-4}, 6.000 \times 10^{-5})^T.$$

7.3.3 预处理共轭梯度算法



对于方程组(2), 如右图所示, 两种算法收敛速度都比较快, 而预处理共轭梯度法更优. 两种方法的解向量均为 $\mathbf{x}^* = (2.625, -1.875, 12.000, -10.000, 3.875, -0.625)^T$, 两种方法并未达到精度要求, 原因在于共轭梯度法和预处理共轭梯度法均达到了迭代终止条件 $\mathbf{r}^{(k)} = \mathbf{0}$ 或 $(\mathbf{p}^{(k)}, \mathbf{A}\mathbf{p}^{(k)}) = 0$. 如预处理共轭梯度法终止条件 $(\mathbf{p}^{(k)}, \mathbf{A}\mathbf{p}^{(k)}) \approx 10^{-53} \approx 0$. 对于这两个病态方程组, 最速下降法迭代10000次均不收敛.